



yarg-sync — the sync client

`yarg-sync` pulls a `yarg-song-server` library into an ordinary local songs folder. **YARG is not modified and does not know the server exists.** It sees files in a folder, which is all it has ever needed to see.

That is the whole point of Phase 2b: shared libraries work today, on stock YARG, on every platform YARG runs on, with no client fork and no risk to anyone's install.

```
yarg-sync -server http://pi.local:8080 -songs "%USERPROFILE%\YARG Songs"
```

Flags

Flag	Default	Meaning
<code>-server</code>	<i>(required)</i>	Base URL of a <code>yarg-song-server</code> .
<code>-songs</code>	<code>./songs</code>	Local folder to sync into.
<code>-dry-run</code>	off	Report what would change; write nothing.
<code>-prune</code>	off	Delete songs the server no longer has.
<code>-version</code>		Print version and exit.

Exit status is `0` only if every song the server offered was fetched and verified. A run that fetched some songs and failed on others exits `1`. A sync that silently skipped half a library is exactly the kind of green this project audits other people's scripts for.

What it will and will not touch

The client writes exactly one shape of file:

```
<chart_hash>.sng          # 40 lowercase hex characters, then ".sng"
```

Anything else in that folder is not ours and is never read, renamed, rewritten or deleted — not by a normal run, and not by `-prune`. That includes a loose song folder the player charted themselves, a `.sng` they named by hand, and a file whose name is nearly ours but not quite. `internal/e2e` (`TestThePlayersOwnSongsAreNeverTouched`) snapshots such a folder, syncs, prunes, and compares every byte.

So it is safe to point `-songs` at a folder the player already uses. That was a design requirement, not a happy accident: a sync tool that can eat somebody's own charts is one nobody should run.

`-prune` is off by default and costs an extra request (a second `POST /api/v1/have` with an empty list, to learn the server's full set). Deleting a player's music because the server was briefly reachable but empty is a failure mode worth paying a round trip to avoid.

Why files are named by chart hash

Song identity in this project is `SHA1(chart file bytes)` and nothing else — the same rule the server indexes by and the same rule YARG uses. Naming the file after it makes the local folder self-describing: the client can take inventory by reading directory entries, with no state file to corrupt, lose or disagree with reality.

It also means the client can check the server's work. Every download lands as `<hash>.sng.part`, is opened and scanned locally, and is only renamed into place if the identity re-derived from the bytes matches the one that was asked for. A truncated transfer, a proxy serving something else, or a server bug all fail closed. `TestCorruptDownloadIsRefused` and `TestWrongSongIsRefused` cover it.

When two packages share a chart

Two folders can hold the same chart with different extras — different album art, a different `charter`, an extra stem. Identity is the chart, so the client wants one copy, and the server refuses to guess: `GET /song/{hash}.sng` answers **300 Multiple Choices** with the candidates. The client then picks the lowest package hash, which is arbitrary but *deterministic*: two machines syncing the same server get the same bytes, which is what makes a shared library shared.

Windows Defender flags the binary (false positive)

Measured on ENG-1, 2026-09-05: Defender quarantines `cmd/yarg-sync` builds as

```
Trojan:Win32/Bearfoos.A!ml (ThreatID 2147731250)
```

`go build ./cmd/yarg-sync` fails with *"the file contains a virus or potentially unwanted software"* before it can link.

This is a machine-learning heuristic (`!ml`) firing on the shape of the program, not on anything in it: a small, unsigned, statically linked Go executable that makes HTTP requests and writes files to disk is what a downloader-dropper looks like to a classifier. It is a well-known false positive for Go binaries. The server binary, which does not download anything, is not flagged.

Notes for whoever hits this next:

- **It is not a CI problem.** The runner is Linux; `release` and the container image build fine. Only a Windows developer build hits it.
- **Test binaries are not flagged**, which is why `internal/e2e` exercises the client in-process instead of executing the built binary. A test that shelled out would be red on the primary development machine for a reason that has nothing to do with this code.
- **The real fix is a code-signing certificate** for the Windows release artifact. Until then, players who download `yarg-sync-windows-amd64.exe` should expect the same warning, and the README says so.
- Do not "fix" this by telling anyone to disable Defender. Submitting the binary to Microsoft's false-positive form is the correct escalation, and signing is the correct solution.

How this is verified

`internal/syncclient` unit-tests the client against HTTP stubs. Stubs prove the client's own logic, but a stub only ever says what its author believed the server says — it cannot catch the two sides disagreeing about the wire.

`internal/e2e` therefore wires the real `httpapi.Server`, the real `library.Build` and the real `packcache` to the real client and runs whole syncs against a real library on disk:

Test	What would have to break for it to fail
<code>TestWholeSyncRoundTrip</code>	Client ends up with every song, each archive readable and each one the song its name claims — identity re-derived from the downloaded bytes.
<code>TestSecondRunDownloadsNothing</code>	Sync is idempotent; an up-to-date client transfers zero song bytes.
<code>TestThePlayersOwnSongsAreNeverTouched</code>	Nothing unmanaged is altered, including across a <code>-prune</code> .
<code>TestDryRunWritesNothing</code>	<code>-dry-run</code> names every song and writes no entries.
<code>TestSharedChartHashSyncsOnce</code>	The 300 Multiple Choices exchange completes and yields one archive.

Each was red-proofed on 2026-09-05 by breaking the code it covers and confirming that test, and only that test, failed:

- `managedName` loosened from `^[0-9a-f]{40}\.sng$` to `\.sng$` → `TestThePlayersOwnSongsAreNeverTouched` alone failed.
- the `-dry-run` guard replaced with `if false` → `TestDryRunWritesNothing` alone failed.

A test that has never been seen to fail is a test whose meaning is unmeasured.